

AI Team OS — Nasıl Çalışır?

Bir hedef cümlesinden merge edilmiş kodlara, gözetimsiz çalışan yapay-zekâ yazılım ekibi · 23 Ağustos 2026

1. Nedir?

AI Team OS, gerçek **Claude Code CLI** ajanlarından kurulu bir yazılım ekibini işleten bir orkestrasyon sistemidir. Siz tek bir **hedef (goal)** cümlesi yazarsınız; sistem geri kalan her şeyi kendi başına yapar:

HEDEF → **PLANLAMA** (yönetici ajan görev grafi çıkarır) → **ÇALIŞMA** (geliştirici ajanlar, izole worktree'lerde) → **DOĞRULAMA** (verify komutları) → **KOD İNCELEMESİ** (bağımsız QA ajanı diff'i yargılar) → **MERGE** (serileştirilmiş kuyruk, rebase + yeniden doğrulama) → **main'de biten iş**

Tanım gereği "bitti" demek: *verify komutları yeşil VE bağımsız bir QA ajanının onayı VE main'e merge edilmiş olmak*. Bir ajanın kendi işine "bitti" demesi asla yeterli değildir — otonom sistemlerin bir numaralı hata kipi tam da budur.

2. Mimari — Büyük Resim

Sistem bir TypeScript monorepo'sudur. Üç ana katman vardır:

Katman	Nerede	Ne yapar
Domain (saf çekirdek)	packages/domain	Hiç yan etkisiz saf fonksiyonlar: görev/koşu durum makineleri, <code>decide()</code> zamanlayıcısı, guardrail değerlendirmesi, merge kuyruğu sıralaması, plan-grafiği ve inceleme-kararı ayrıştırıcıları (Zod şemalı). Tamamı bağımsız test edilir.
Orkestratör (daemon)	apps/orchestrator	Her saniye bir tick atar: dünyayı veritabanından yeniden okur, <code>decide()</code> 'a sorar, koşuları başlatır/durdurur, planlama-inceleme-merge geçişlerini (pass) koşturur, guardrail süpürmesini yapar.
Web arayüzü	apps/web (Next.js)	Overview, görev panosu, canlı etkinlik akışı ve bağımlılık grafiği. Canlılık SSE ile: her olay arayüze anında düşer. Arayüz asla "iyimser" değildir — gerçeği her zaman veritabanı anlık görüntüsü söyler.

Aradaki tutkal: **Postgres** (tek gerçek kaynağı), **append-only olay günlüğü** (`ExecutionEvent` — sistemde olan her şeyin değiştirilemez kaydı; 28 olay tipi, tek yazım kapısı) ve **LISTEN/NOTIFY** (bildirim = uyandırma; her tick dünyayı yine de baştan okur, bu yüzden kaçan bildirim doğruluğu değil yalnız gecikmeyi etkiler).

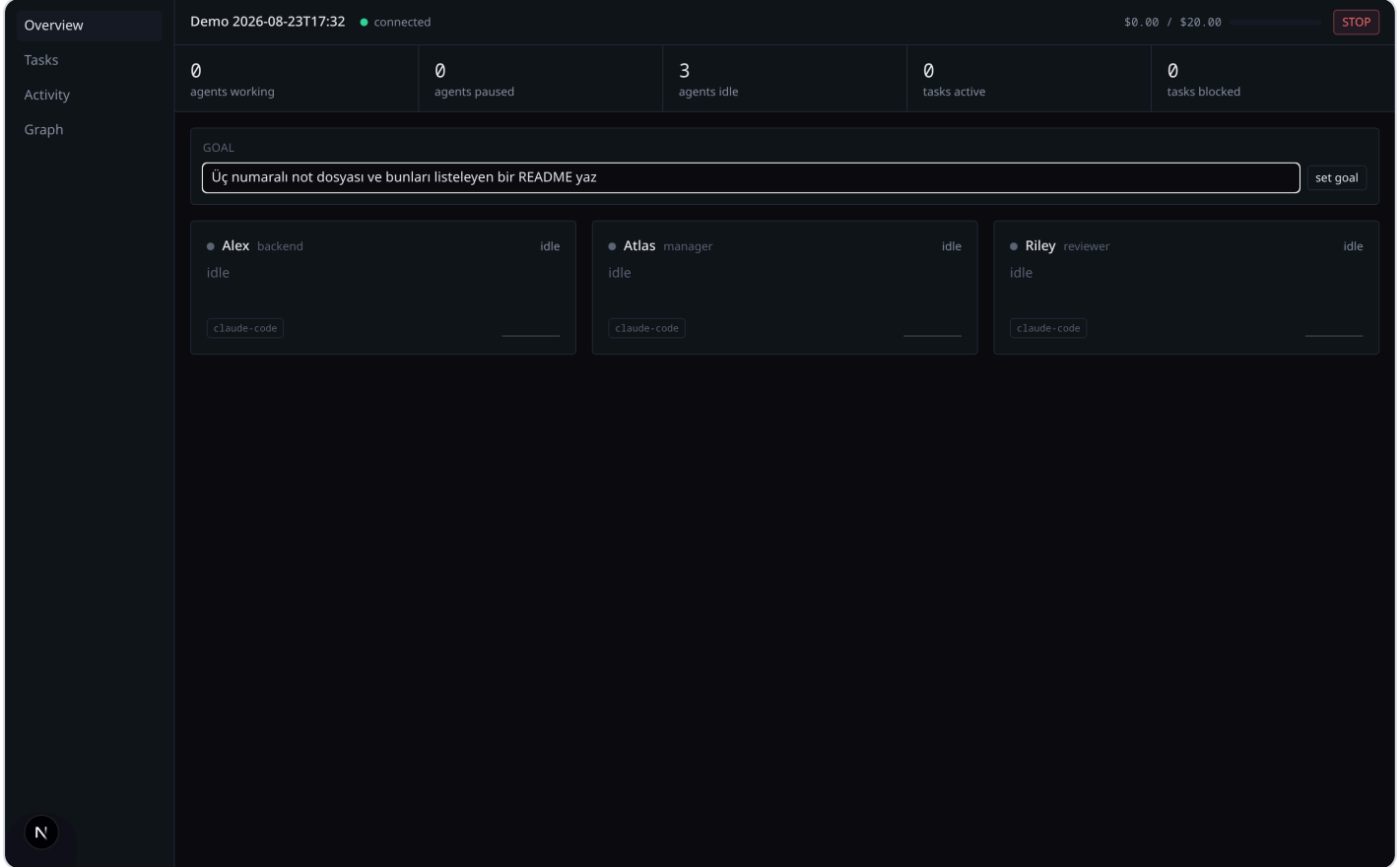
Tick'in geçiş sırası: zamanla (schedule) → devam ettir (resume) → **planla** → **incele** → **merge et** → guardrail süpürmesi. Her geçiş, bir öncekinin göremeyeceği durumlara tepki verir; hepsi çökmeye dayanıklıdır (claim kolonları, koşullu yazımlar).

3. Akış, Adım Adım (Canlı Ekran Görüntüleriyle)

Aşağıdaki kareler bu doküman için gerçek sistemden alındı: `npm run demo` + sahte Claude CLI (para harcamayan prova modu) ile canlı bir turdur.

3.1 · Her şey bir hedefle başlar

Demo açıldığında pano bilerek bomboştur: görev yok, plan yok. Overview sayfasında üç ajan (Atlas-yönetici, Alex-geliştirici, Riley-inceleyci) boşa bekler. Hedefinizi GOAL formuna yazarsınız:



Görüntü 1. Boş pano, boştaki ekip ve doldurulmuş hedef formu: "Üç numaralı not dosyası ve bunları listeleyen bir README yaz". Sağ üstte bütçe barı (\$0/\$20) ve acil durdurma (STOP) düğmesi.

3.2 · Planlama: yönetici ajan hedefi görev grafına çevirir

Hedef kaydedilir kaydedilmez daemon'ın **planlama geçişi** devreye girer: pano boş + hedef var + `manager` rollü ajan boşa → Atlas için bir **planlama koşusu** başlar. Atlas depoyu okur ve son mesajında tek bir JSON görev grafi üretir. Bu graf **Zod ile doğrulanır**: 1-20 görev, benzersiz anahtarlar, var olan bağımlılıklar, **çevrimsizlik** (Kahn algoritması). Serbest metin asla veritabanına ulaşmaz; geçersiz çıktı koşuyu "failed" yapar ve sistem en fazla 2 kez yeniden dener.

OverviewTasksActivityGraph

Demo 2026-08-23T17:32connected\$0.00 / \$20.00STOP

1agents working

0agents paused

2agents idle

0tasks active

0tasks blocked

GOALÜç numaralı not dosyası ve bunları listeleyen bir README yaz

Alexbackendidleidleclaude-code

Atlasmanagerworkingidleclaude-code

Rileyrevieweridleidleclaude-code

Görüntü 2. Hedef kaydedildi; Atlas (manager) "working" — planlama koşusu uçuşta. Görev henüz yok: koşunun kendisi görevsizdir (task-less run).

Geçerli graf **tek bir veritabanı transaction'ında** görevlere ve bağımlılık kayıtlarına dönüşür — yarım pano diye bir şey olamaz. Ardından olay günlüğüne görev başına `task.created` ve bir `workspace.plan_created` düşer: planın insan-görünür kaydı.

OverviewTasksActivityGraph

Demo 2026-08-23T17:32connected\$0.00 / \$20.00STOP

RunsTool callsTasksInterventionsGuardrailsWorkspaceAdvanced

AGENTS

AlexAtlasRiley

TASKS

Document and polishExpose the API!Write the feature core

17:34:11humangoal set — Üç numaralı not dosyası ve bunları listeleyen bir README yaz

17:34:11agentAtlasrun started

17:34:12agentAtlasWrite — Write /fake/claude-workdir/complete/alpha.txt

17:34:17agentAtlasCreated '/fake/claude-workdir/complete/alpha.txt' containing 'hello'.

17:34:18agentAtlasBash — Bash echo hello > /fake/claude-workdir/complete/gamma.txt && cat /fake/claude-workdir...

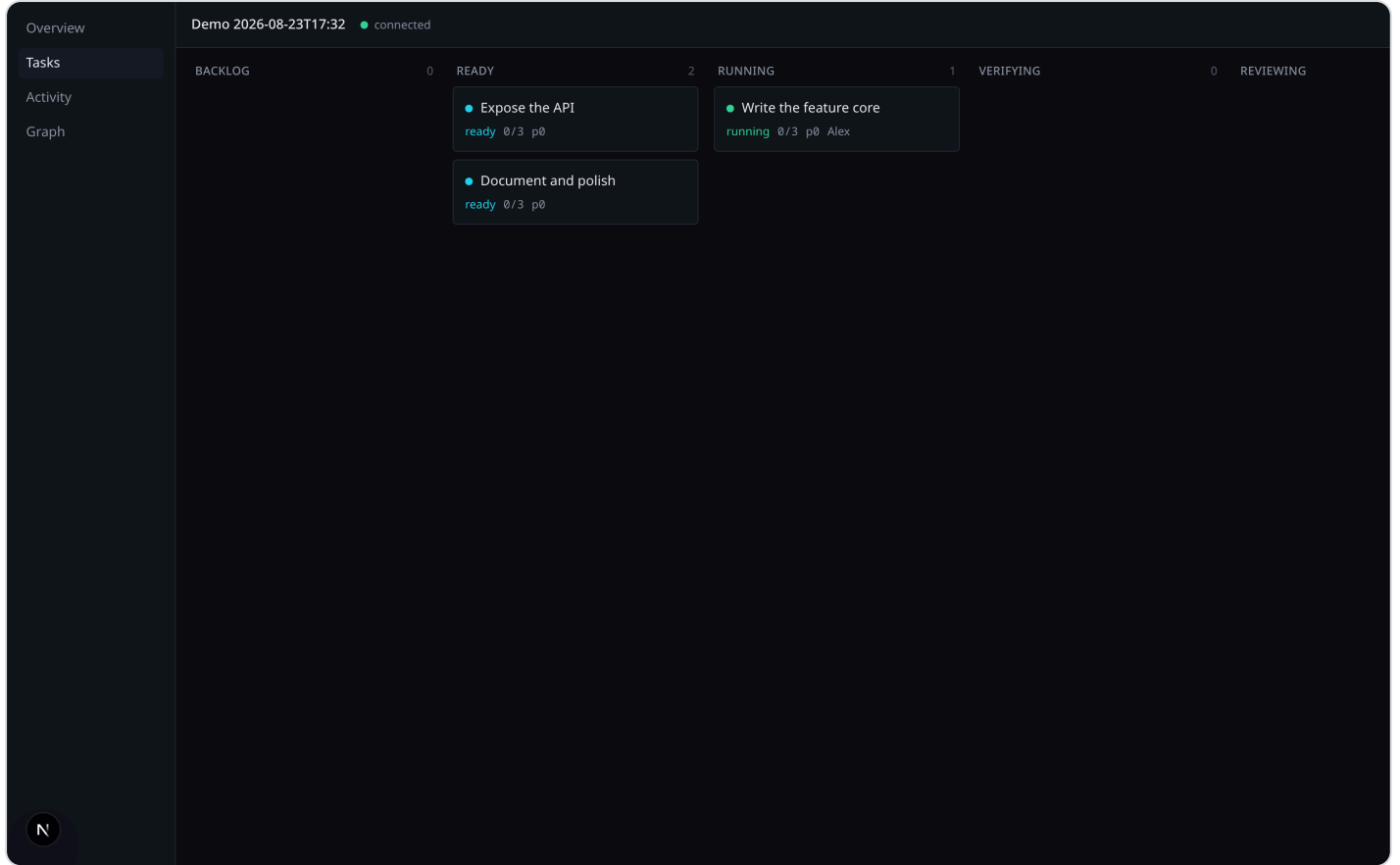
17:34:23agentAtlasHere is the plan. {"tasks":[{"key":"core","title":"Write the feature core","description":"Implement the core module the ...

17:34:27systemAtlasrun succeeded — 3 turns · \$0.21

Görüntü 3. Etkinlik akışı: "goal set" kartı ve "planned 3 tasks" kartı — plan, çalışma başlamadan görünür durumda.

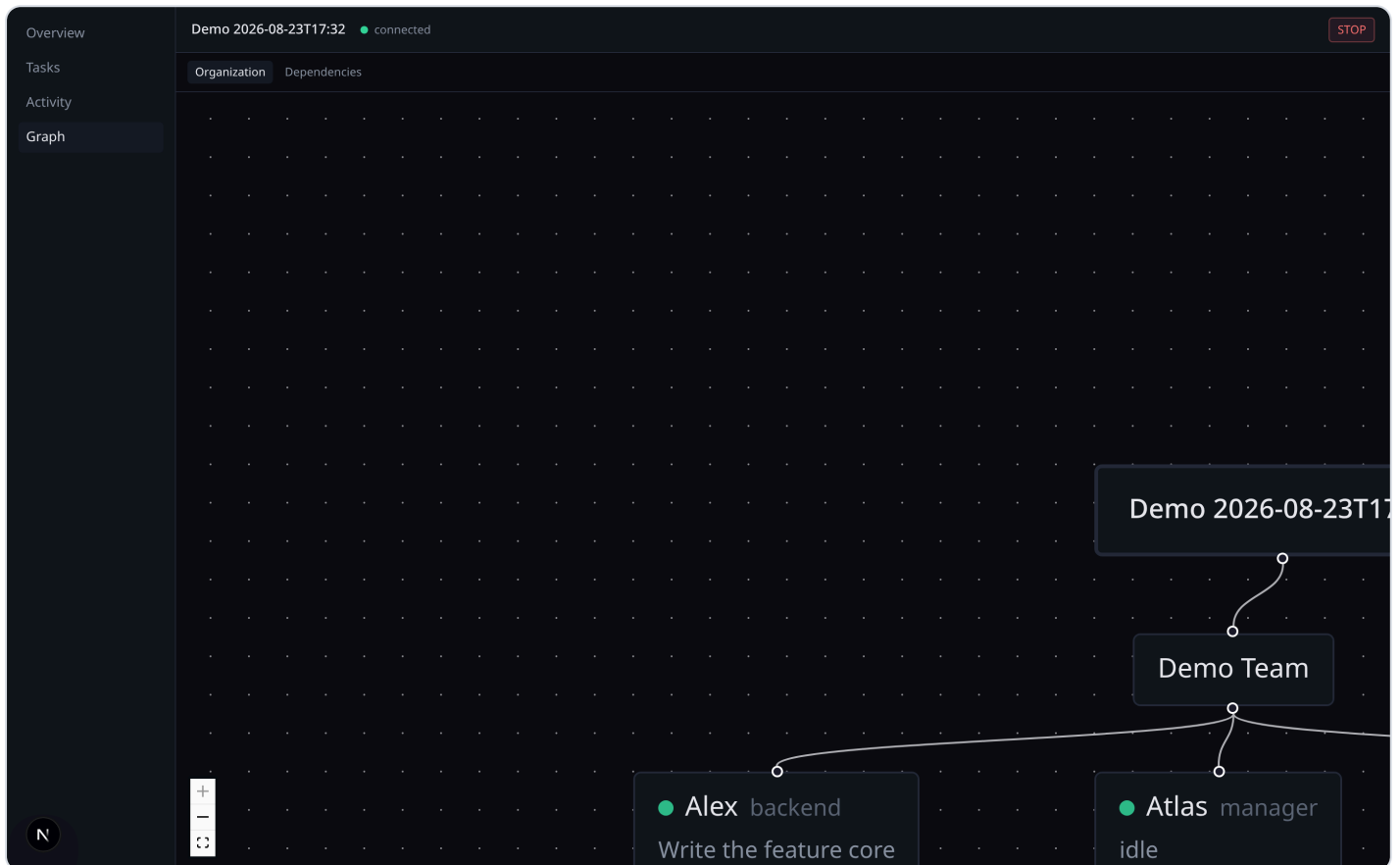
3.3 · Pano dolar, kök görev hemen çalışmaya başlar

Planlanan görevler **anında zamanlanabilir** doğar (insan onay kapısı yok — görünürlük güvencedir, kapı değil). Zamanlayıcı yalnız *bağımlılıkları tamamlanmış* görevleri başlatır: zincirin kökü hemen Alex'e atanır, diğerleri sırasını bekler.



Görüntü 4. Plan panoya indi: "Write the feature core" RUNNING (Alex), bağımlıları READY'de bekliyor.

Her çalışma koşusu **izole bir git worktree**'sinde, kendi dalında yaşar. Koşu biterken sistem workspace'in **verify komutlarını** o worktree'de koşturur; yeşilse görev **reviewing** 'e geçer.



Görüntü 5. Graph sayfası: planın çıkardığı bağımlılık zinciri (core → api → polish) canlı durum renkleriyle.

3.4 · Bağımsız kod incelemesi ve serileştirilmiş merge

İnceleme geçişi, `reviewing` 'deki her görev için `reviewer` rollü ajanla (Riley) bir QA koşusu başlatır — korunmuş `worktree`'de, diff'i yargılar (yeniden inşa etmez). Kararı yine katı bir JSON sözleşmesidir:

`{"verdict": "approve|reject", "reason": "..."} .` Onay görevi `merging` 'e taşır; ret, olağan yeniden-çalışma (rework) yoluna sokar (görev başına en çok 3 deneme).

Merge geçişi bilinçli olarak serileştirilmiştir: tick başına en çok BİR görev, bir claim kolonu üzerinden koşullu yazımla sahiplenilir (çift yürütme imkânsız). Sıra, onay olaylarının zamanına göre FIFO'dur. Sahiplenilen görev: dal `main` üzerine **rebase** edilir → rebase sonucu **yeniden verify** edilir (asıl kapı budur: iki ayrı yeşil dal birlikte kırmızı olabilir) → görev-anahtarlı `--no-ff` merge commit'i atılır. Çakışma ya da kırmızı verify görevi rework'a döndürür; aynı görev iki kez merge'de kalırsa workspace guardrail ile durdurulur.

3.5 · Acil durdurma: tek düğme, her şey durur — ve temiz geri dönüş

Sağ üstteki STOP, iki aşamalı onaylıdır (yanlışlıkla basılamaz):

Overview

Tasks

Activity

Graph

Demo 2026-08-23T17:32 connected

\$0.63 / \$20.00 stop everything cancel

1agents working

0agents paused

2agents idle

3tasks active

0tasks blocked

GOAL

Üç numaralı not dosyası ve bunları listeleyen bir README yaz

Alex backend

idle

idle

claude-code

Atlas manager

idle

idle

claude-code

Riley reviewer

working

Write the feature core

Bash echo hello > /fake/claude-workdir/complete/g_

claude-code

Görüntü 6. STOP'a basıldı; düğme yerinde "stop everything / cancel" onayı açıldı.

Onayla birlikte: workspace **halt** kolonu ilk-yazan-kazanır kuralıyla set edilir (zamanlama o an durur), ve **aktif her koşuya** duraklatma isteği dağıtılır. Kırmızı şerit her sayfada görünür:

Overview

Tasks

Activity

Graph

Demo 2026-08-23T17:32 connected

\$0.63 / \$20.00 STOP

workspace halted: emergency stop by web operator — retract with `clear-halt` (CLI)

0agents working

1agents paused

2agents idle

3tasks active

0tasks blocked

GOAL

Üç numaralı not dosyası ve bunları listeleyen bir README yaz

Alex backend

idle

idle

claude-code

Atlas manager

idle

idle

claude-code

Riley reviewer

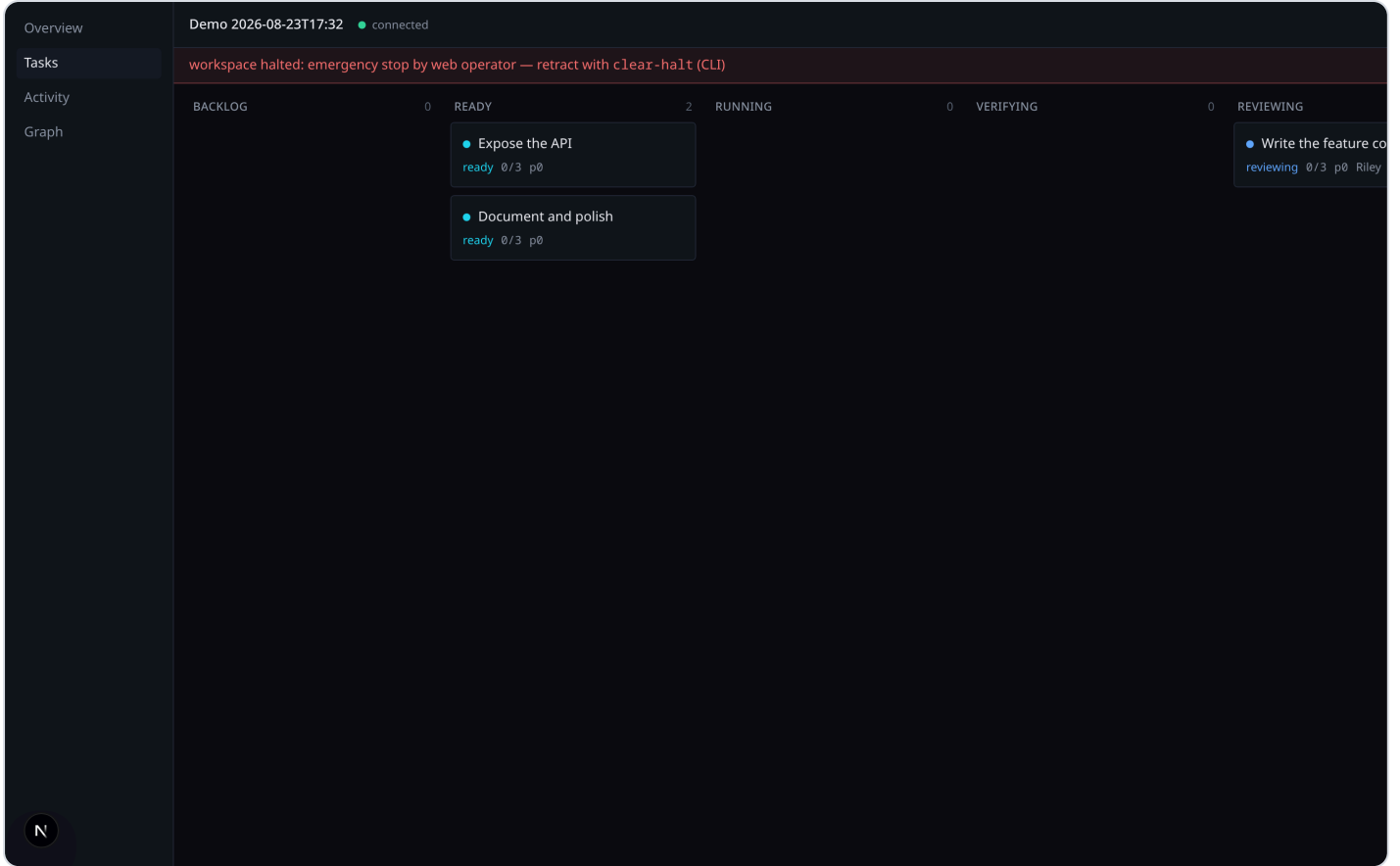
pausing

Write the feature core

Bash echo hello > /fake/claude-workdir/complete/g_

claude-code

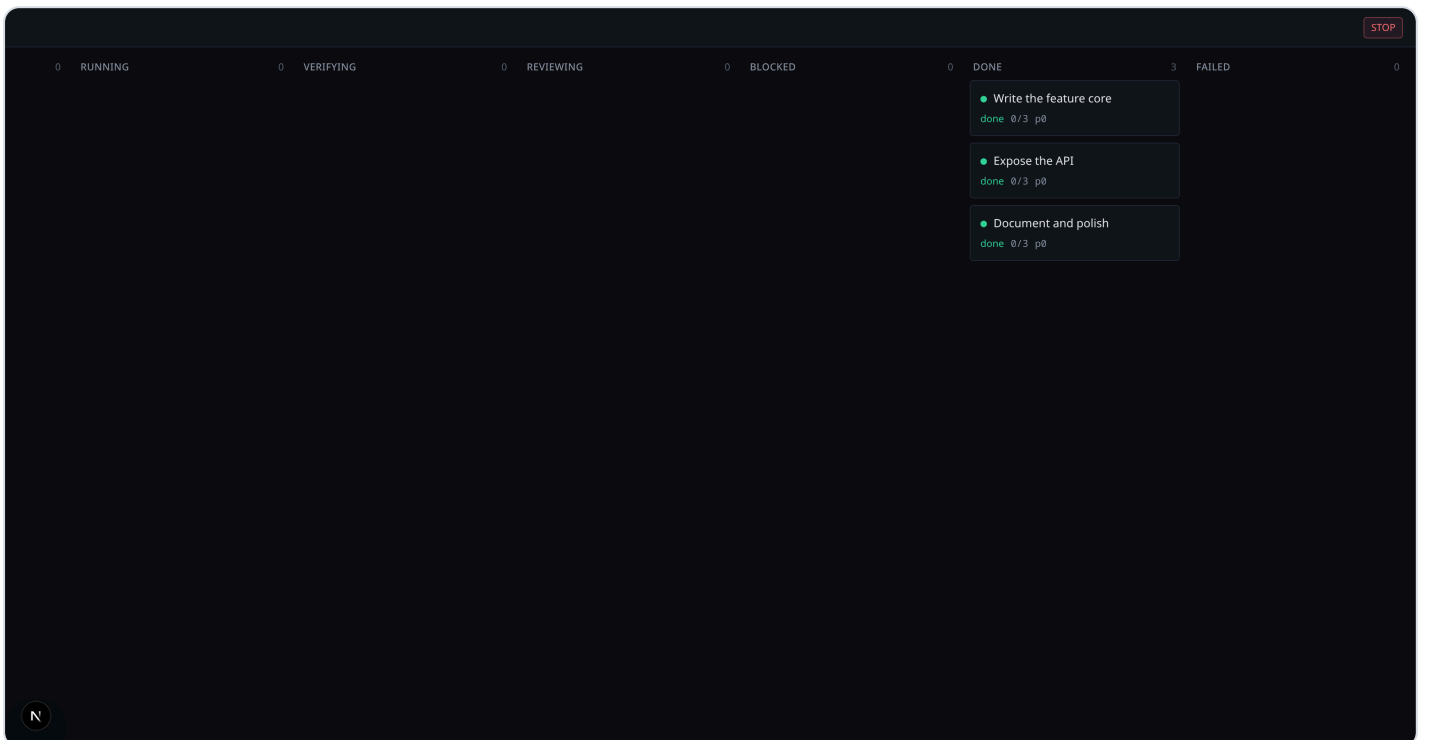
Görüntü 7. "workspace halted: emergency stop by web operator". Riley'nin canlı inceleme koşusu "pausing" durumuna geçmiş; sayaçlar durumu anlatıyor. Bütçe barı o ana dek harcananı gösteriyor (\$0.63).



Görüntü 8. Aynı şerit görev panosunda — operatör hangi sayfadaysa orada.

Geri dönüş de aynı derecede temizdir: `clear-halt` zamanlamayı serbest bırakır, duraklamış koşullar `resume` ile kaldıkları oturumdan devam eder. (Bu turda da öyle yaptık; akış kaldığı yerden sürüp bitti.)

3.6 • Bitiş: hedef, main'de üç merge commit'i olarak



Görüntü 9. Üç görev de DONE kolonunda.

Deponun `main` dalındaki kanıt — görev-anahtarlı merge commit'leri (her biri tek `git revert -m 1` ile geri alınabilir):

```
8a6c206 merge(T-b88b5140): Document and polish
53b5195 merge(T-d32f9386): Expose the API
929b15e merge(T-e605bb3d): Write the feature core
9f355e6 initial
```

4. Guardrail'ler — Otonominin Emniyet Kemer

Tam otonomi, guardrail'leri opsiyonel değil **zorunlu** kılar. Hepsi workspace ayarıdır ve daemon tarafından her tick'te uygulanır:

Guardrail	Davranış	Varsayılan
Bütçe tavanı	%80'de bir kez uyarı olayı; aşımında zamanlama durur ve aktif koşular duraklatılır	\$20 / workspace
Eşzamanlılık	Aktif koşu tavanı — workspace başına ve tüm sistemde	3 / workspace, 6 global
Koşu zaman aşımı	Duvar-saati sınırını aşan koşu iptal edilir (guardrail süpürmesi)	30 dk
Araç çağırısı tavanı	Sınırı aşan koşu iptal edilir	200 çağrı / koşu
Deneme tavanı	Rework sınırı; aşan görev failed olur	3 deneme / görev
Devre kesici	Üst üste başarısız koşular workspace'i durdurur	3 ardışık
Acil durdurma	Tek anahtar: zamanlama durur + tüm aktif koşulara duraklatma (Bölüm 3.5)	—
İzinler	Ajan başına araç izin/yasak listesi, adaptör seviyesinde (UI'da değil)	rol başına

Escalasyon dili tektir: sıra dışı her durum, olay günlüğüne bir `guardrail.tripped` kaydı düşürür (bütçe uyarısı, plancı/inceleyci yokluğu, tekrarlanan merge hatası...). Tekrar eden durumlar "bir kez" ilkesiyle kaydedilir — operatör gürültüye boğulmaz.

5. Kendiniz Çalıştırın

```
# 1) Veritabanı (bir kere):      docker compose up -d
# 2) Web arayüzü (bir terminal):  npm run web
# 3) Demo (ikinci terminal):     npm run demo
#   → yazdırılan Overview URL'ini açın, hedefi forma yazın, izleyin.

# Para harcamadan prova (sahte Claude CLI ile):
AITEAMOS_CLAUDE_BIN=node \
AITEAMOS_CLAUDE_ARGS="$(pwd)/packages/providers/test/fake-claude.mjs --fixture m8-flow" \
npm run demo
```

Üç ölçülü kapı script'i, sistemin ana iddialarını tek komutla kanıtlar: `npm run gate:m8-plan` ("bir hedef → merge edilmiş dallar, gözetimsiz"), `npm run gate:m8a-merge` ve `npm run gate:m8a-estop` (acil durdurma + temiz dönüş).

6. Küçük Sözlük

Terim	Anlamı
Tick	Daemon'ın kalp atışı (~1 sn): dünyayı oku, karar ver, geçişleri koştur.
Pass (geçiş)	Tick içindeki tek amaçlı adım: planlama, inceleme, merge, süpürme.
Worktree	Her koşunun izole git çalışma kopyası; işler birbirine bulaşmaz.
Task-less run	Görevi olmayan koşu — planlama koşusu; hedefe çalışır, göreve değil.
Claim kolonu	"Önce ben aldım" koşullu yazımı; çökme ve yarışlara karşı çift-yürütme sigortası.
Olay günlüğü	Append-only <code>ExecutionEvent</code> tablosu; arayüzün ve denetimin tek kaynağı.

Bu doküman ve içindeki tüm ekran görüntüleri, sistemin kendisi canlı sürülerek üretildi (23 Ağustos 2026). Daha derin okuma: `docs/architecture.md`, `docs/domain-model.md`, `docs/event-model.md` ve `docs/decisions/` (ADR 0001–0006).